

QUESTION 32: LORA-BASED FINE-TUNING

32. LoRA-Based Fine-Tuning A healthcare organization wants to adapt an existing LLM to generate standardized medical summaries but has limited computational resources.

Task: Explain how LoRA/PEFT could be used instead of full fine-tuning. Describe the expected workflow and advantages of the approach.

SCENARIO

A healthcare organization wants to adapt an existing Large Language Model (LLM) to generate standardized medical summaries but has limited computational resources.

1. PROBLEM ANALYSIS

OBJECTIVE

Develop a function-calling system that configures LoRA (Low-Rank Adaptation) fine-tuning for a pretrained LLM and returns the training configuration in a structured JSON format.

Instead of performing full model fine-tuning, the application uses PEFT (Parameter-Efficient Fine-Tuning) to train only lightweight LoRA adapter layers.

2. FUNCTION CALLING SCHEMA

FUNCTION NAME

`configure_lora_finetuning()`

PURPOSE

Configure LoRA fine-tuning parameters for adapting a pretrained LLM to generate standardized medical summaries.

PARAMETERS

Parameter	Type	Required	Description
model_name	string	Yes	Name of pretrained LLM
dataset_path	string	Yes	Dataset file location
learning_rate	float	Yes	Model learning rate
epochs	integer	Yes	Number of training epochs
batch_size	integer	Yes	Batch size
lora_rank	integer	Yes	Rank of LoRA adapters
lora_alpha	integer	Yes	LoRA scaling factor
target_modules	array	Yes	Transformer layers to adapt

3. JSON FUNCTION SCHEMA

```
{  
  "name": "configure_lora_finetuning",  
  "description": "Configure LoRA fine-tuning parameters.",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "model_name": {  
        "type": "string",  
        "description": "Pretrained LLM"  
      },  
      "dataset_path": {
```

```
    "type": "string",
    "description": "Dataset location"
  },
  "learning_rate": {
    "type": "number"
  },
  "epochs": {
    "type": "integer"
  },
  "batch_size": {
    "type": "integer"
  },
  "lora_rank": {
    "type": "integer"
  },
  "lora_alpha": {
    "type": "integer"
  },
  "target_modules": {
    "type": "array",
    "items": {
      "type": "string"
    }
  }
}
```

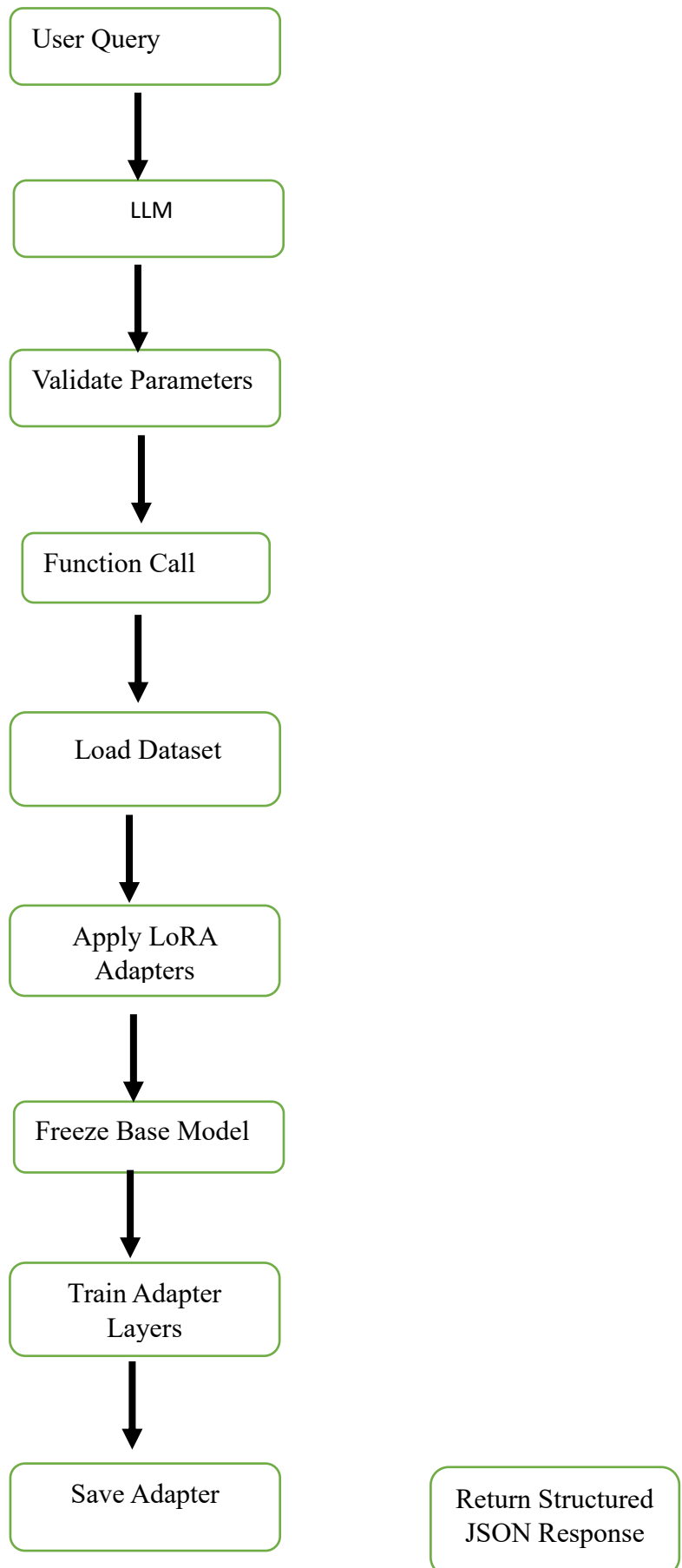
```
    },  
    "required": [  
        "model_name",  
        "dataset_path",  
        "learning_rate",  
        "epochs",  
        "batch_size",  
        "lora_rank",  
        "lora_alpha",  
        "target_modules"  
    ]  
}  
}
```

4. STRUCTURED OUTPUT (JSON SCHEMA)

```
{  
    "status": "success",  
    "model": "Llama-3.2",  
    "method": "LoRA",  
    "epochs": 3,  
    "batch_size": 4,  
    "training_status": "Completed",  
    "adapter_saved": true,  
    "message": "LoRA fine-tuning configuration completed successfully."  
}
```

}

5. FUNCTION CALLING WORKFLOW





6. PYTHON IMPLEMENTATION

```
import json

def configure_lora_finetuning(
    model_name,
    dataset_path,
    learning_rate,
    epochs,
    batch_size,
    lora_rank,
    lora_alpha,
    target_modules
):
    if dataset_path == "":
        return {
            "status": "error",
            "message": "Dataset path is required."
        }

    if epochs <= 0:
        return {
            "status": "error",
            "message": "Epochs must be greater than zero."
        }

    result = {
```

```

        "status": "success",

        "model": model_name,

        "method": "LoRA",

        "learning_rate": learning_rate,

        "epochs": epochs,

        "batch_size": batch_size,

        "lora_rank": lora_rank,

        "lora_alpha": lora_alpha,

        "target_modules": target_modules,

        "training_status": "Completed",

        "adapter_saved": True,

        "message": "LoRA fine-tuning configuration completed successfully."

    }

    return result

response = configure_lora_finetuning(

    model_name="Llama-3.2",

    dataset_path="medical_dataset.csv",

    learning_rate=0.0002,

    epochs=3,

    batch_size=4,

    lora_rank=8,

    lora_alpha=16,

    target_modules=["q_proj", "v_proj"]

)

```

```
print(json.dumps(response, indent=4))
```

7. TEST CASES

TEST CASE 1 (NORMAL)

INPUT

Model = Llama-3.2

Dataset = medical_dataset.csv

Epochs = 3

Batch Size = 4

OUTPUT

```
{  
  "status": "success",  
  "model": "Llama-3.2",  
  "method": "LoRA",  
  "training_status": "Completed",  
  "adapter_saved": true,  
  "message": "LoRA fine-tuning configuration completed successfully."  
}
```

TEST CASE 2 (MISSING DATASET)

INPUT

Dataset Path = ""

OUTPUT

```
{  
  "status": "error",
```



```
"message": "Dataset path is required."
}
```

TEST CASE 3 (INVALID EPOCHS)

INPUT

Epochs = -2

OUTPUT

```
{
  "status": "error",
  "message": "Epochs must be greater than zero."
}
```

8. EDGE CASES HANDLED

- Missing dataset path
- Invalid number of epochs
- Incorrect parameter values
- Empty required fields
- Invalid data types (can be validated before function execution)
- Prevents invalid function calls by returning structured error messages

9. BRIEF DOCUMENTATION

FUNCTION PURPOSE

The `configure_lora_finetuning()` function accepts LoRA training parameters, validates the inputs, and returns a structured JSON response describing the fine-tuning configuration.

EXPECTED INPUT

- Pretrained model name
- Medical dataset path
- Learning rate
- Number of epochs
- Batch size
- LoRA rank
- LoRA alpha
- Target transformer modules

EXPECTED OUTPUT

A structured JSON object containing:

- Training status
- Model name
- LoRA configuration
- Adapter status
- Success or error message

10. DESIGN JUSTIFICATION

- **Function Calling:** Ensures the LLM executes a well-defined function with validated parameters instead of producing arbitrary actions.
- **Structured Output:** JSON responses provide a consistent format that applications can easily parse and integrate.
- **Input Validation:** Required fields and error handling prevent invalid requests and improve robustness.

- **LoRA/PEFT:** Trains only lightweight adapter layers, reducing GPU memory, training time, and storage while preserving the original model.

11. ADVANTAGES OF LORA/PEFT

- Reduces GPU memory usage
- Faster training than full fine-tuning
- Lower computational cost
- Smaller storage requirements
- Preserves the pretrained model
- Supports multiple task-specific adapters
- Easy deployment and maintenance